



POLITECHNIKA POZNAŃSKA

WYDZIAŁ INFORMATYKI I TELEKOMUNIKACJI
Instytut Informatyki

Praca dyplomowa magisterska

ODKRYWANIE ZALEŻNOŚCI MIĘDZY OBIEKTAMI W BAZIE DANYCH

Maria Nowicka, 151851

Promotor
dr hab. inż. Robert Wrembel, prof. PP.

POZNAŃ 2026

Tutaj będzie karta pracy dyplomowej;
oryginał wstawiamy do wersji dla archiwum PP,
w pozostałych kopiach wstawiamy ksero.

Streszczenie

[Streszczenie zostanie uzupełnione po ukończeniu rozdziałów 4 i 5.]

Abstract

[The abstract will be completed once chapters 4 and 5 are finished.]

Spis treści

Streszczenie	iii
Abstract	I
1 Wstęp	1
1.1 Cel i zakres pracy	1
1.2 Struktura pracy	2
2 Podstawy teoretyczne i przegląd literatury	3
2.1 Data lineage i data provenance	3
2.2 Struktura grafu lineage	3
2.3 Niekompletność grafów lineage — broken lineage	4
2.4 Detekcja węzłów o zerwanym lineage	4
2.5 Metryki ewaluacji	5
2.6 Wyniki systematycznego przeglądu literatury	5
2.6.1 Prace bazowe: metoda kontrastowa i zbiór danych	6
2.6.2 Metody pokrewne odkrywania zależności lineage	6
2.6.3 Luka badawcza	8
2.6.4 Prace kontekstowe	8
3 Problem i sposób jego rozwiązania	9
3.1 Sformułowanie problemu	9
3.2 Zbiór danych DLG-DG-23	9
3.2.1 Charakterystyka grafów	10
3.3 Metodyka symulacji broken lineage przez usuwanie jobów	10
3.3.1 Kontrola przecieku: wykluczenie węzłów izolowanych	11
3.4 Architektura rozwiązania	11
3.4.1 Moduł wczytywania danych	11
3.4.2 Moduł symulacji (job_removal)	12

3.4.3	Moduł ewaluacji (node_metrics)	12
3.5	Zaimplementowane metody detekcji	12
3.5.1	Heurystyki strukturalne	12
3.5.2	Klasyczne metody uczenia maszynowego	13
3.5.3	Autorski detektor — LineageDetector	14
3.6	Dostępność implementacji i pochodzenie algorytmów	15
4	Ocena rozwiązania	17
4.1	Badanie wstępne: predykcja krawędzi	17
4.2	Wyniki detekcji węzłów o zerwanym lineage	17
4.3	Kontrola przecieku i analiza wrażliwości	17
4.4	Analiza wyników	17
5	Uwagi końcowe	18
5.1	Podsumowanie	18
5.2	Kierunki dalszych badań	18
	Literatura	19

Rozdział 1

Wstęp

Współczesne systemy bazodanowe przetwarzają dane w złożonych potokach transformacji. W środowiskach korporacyjnych dane są wielokrotnie agregowane i przekształcane przez sekwencje zapytań, a wyniki pośrednie przechowywane w tabelach tymczasowych. Tymczasowość tych obiektów sprawia, że po zakończeniu procesu przetwarzania ślad po wykonanych transformacjach zanika, a zatem niemożliwe staje się ustalenie, skąd pochodzi dana wartość w tabeli wynikowej ani jakie operacje ją wytworzyły. Śledzenie takich zależności jest przedmiotem dziedziny określanej w literaturze jako *data lineage* lub *data provenance* [11, 13]. Brak pełnej informacji o przepływie danych utrudnia audyt i kontrolę jakości danych.

Formalnie problem ten można wyrazić w kategoriach teorii grafów: graf zależności (ang. *lineage graph*) reprezentuje przepływ danych między obiektami bazy takimi jak tabele, widoki, zapytania oraz ich atrybutami. Gdy pewne węzły lub krawędzie zostaną utracone (np. z powodu usunięcia tabeli tymczasowej lub zastosowania funkcji definiowanej przez użytkownika), mówi się o zjawisku *broken lineage* [2].

Celem niniejszej pracy jest *detekcja obiektów bazodanowych o zerwanym lineage* — tabel, które w wyniku broken lineage utraciły producenta lub konsumenta danych. W odróżnieniu od dotychczasowych prac, które modelują broken lineage jako predykcję konkretnych brakujących krawędzi, w pracy sformułowano zadanie na poziomie *węzłów*: uszeregowanie tabel według prawdopodobieństwa, że ich lineage jest niekompletny. Podejście to jest komplementarne wobec predykcji krawędzi i może ją poprzedzać: najpierw wskazuje się podejrzane tabele, a dopiero potem można próbować odtwarzać ich zależności.

W sektorze finansowym banki są zobowiązane do wykazania, które transakcje źródłowe złożyły się na poszczególne pozycje raportów kwartalnych; przerwanie grafu zależności w warstwie ETL (*Extract, Transform, Load*) uniemożliwia przeprowadzenie takiego audytu [2]. Podobne konsekwencje niesie Rozporządzenie o Ochronie Danych Osobowych (RODO): brak pełnego grafu lineage sprawia, że dotarcie do wszystkich kopii danych konkretnej osoby jest czasochłonne lub niemożliwe, co może skutkować nałożeniem kar finansowych [2].

1.1 Cel i zakres pracy

Celem pracy jest zbadanie, w jakim stopniu *sama topologia* grafu lineage pozwala wskazać tabele o zerwanej ciągłości pochodzenia danych, bez dostępu do semantyki nazw obiektów, schematów bazy czy ontologii. Zadanie sformułowano jako ranking węzłów typu *Data Table* według prawdopodobieństwa, że tabela została dotknięta zjawiskiem broken lineage; tabele takie nazywane są dalej *zainfekowanymi* (ang. *infected*). Zakres obejmuje:

- metodykę symulacji broken lineage przez usuwanie zadań (jobów) i wyznaczanie zbioru tabel zainfekowanych jako etykiet referencyjnych,
- porównanie trzech klas metod detekcji: heurystyk strukturalnych, klasycznych klasyfikatorów uczenia maszynowego opartych na cechach pozycji węzła w grafie DAG oraz autorskiego detektora *LineageDetector*,
- ewaluację w protokole *indukcyjnym* cross-graph (trenowanie na części grafów, testowanie na grafach niewidzianych),
- analizę wrażliwości wyniku na odsetek usuwanych jobów oraz kontrolę przecieku wynikającego z trywialnie izolowanych węzłów.

Eksperymenty przeprowadzono na zbiorze danych DLG-DG-23, obejmującym 18 zanonimizowanych grafów lineage ze środowiska Huawei Cloud [4].

Zakres pracy nie obejmuje odtworzenia rzeczywistego schematu bazy danych, analizy treści zanonimizowanych danych ani odtwarzania konkretnych brakujących krawędzi. Predykcja krawędzi została zachowana jako *badanie wstępne*, którego wyniki motywują zmianę ujęcia problemu na poziom węzłów (sekcja 4.1).

1.2 Struktura pracy

Praca składa się z pięciu rozdziałów. Rozdział 2 zawiera przegląd pojęć podstawowych (data lineage, data provenance, broken lineage oraz detekcja anomalii w grafach), a także omówienie wybranych prac z literatury i stosowanych metryk ewaluacji. Rozdział 3 opisuje problem badawczy, metodykę symulacji broken lineage przez usuwanie jobów, architekturę zaproponowanego rozwiązania oraz zaimplementowane algorytmy detekcji. Rozdział 4 prezentuje wyniki eksperymentów przeprowadzonych na zbiorze DLG-DG-23 wraz z ich analizą. Rozdział 5 zawiera wnioski końcowe, podsumowanie uzyskanych wyników oraz propozycje dalszych kierunków badań.

Rozdział 2

Podstawy teoretyczne i przegląd literatury

Niniejszy rozdział wprowadza pojęcia podstawowe niezbędne do zrozumienia dalszych części pracy. W sekcji 2.1 zdefiniowano pojęcia *data lineage* i *data provenance*. Sekcja 2.2 opisuje strukturę grafów *lineage*. Sekcja 2.3 omawia przyczyny niekompletności grafów, a sekcja 2.4 przedstawia zadanie detekcji węzłów o zerwanym *lineage*. Sekcja 2.5 opisuje metryki ewaluacji, a sekcja 2.6 zawiera przegląd wybranej literatury.

2.1 Data lineage i data provenance

Pojęcia *data lineage* i *data provenance* są w literaturze stosowane zarówno zamiennie, jak i z różnieniem semantycznym [11]. W węższym ujęciu *data lineage* odnosi się do śledzenia przepływu informacji przez kolejne etapy przetwarzania, opisuje, przez jakie transformacje przeszły dane i jak zmieniała się ich postać. Z kolei *data provenance* kładzie nacisk na pochodzenie i autentyczność danych, rejestruje, skąd dane zostały pobrane i kto je wytworzył. W praktyce granica między tymi pojęciami jest płynna i większość opracowań traktuje je jako synonimy. Takie podejście przyjęto również w niniejszej pracy.

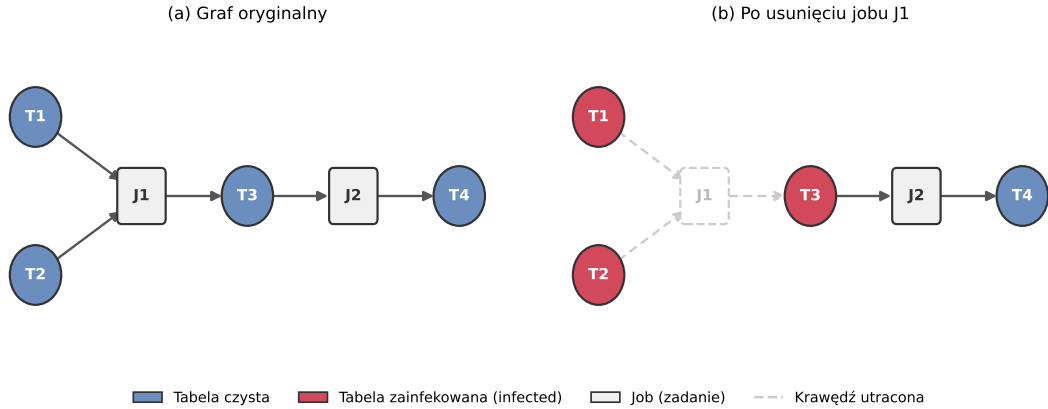
Znajomość historii danych ma istotne znaczenie praktyczne. W systemach raportowych możliwość odtworzenia ścieżki transformacji pozwala zlokalizować źródło błędnych wartości, bez tej informacji ręczna weryfikacja wielkich zbiorów danych jest nieefektywna. Innym przykładem jest bezpieczne usuwanie obiektów bazy: tabela pozornie niepowiązana z żadnym procesem może w rzeczywistości dostarczać danych do raportów końcowych, a jej usunięcie spowoduje utratę danych bez widocznego komunikatu błędu [2].

2.2 Struktura grafu lineage

Graf *lineage* reprezentuje przepływ danych między obiektami bazy danych. Węzłami są obiekty przetwarzania: tabele i widoki przechowujące dane, zadania (zapytania SQL, procesy ETL, ang. *Extract, Transform, Load*) je przekształcające, oraz atrybuty (pola) tabel. Krawędzie skierowane wyrażają przepływ danych — z tabeli do zadania, które ją czyta, oraz z zadania do tabeli, którą zapisuje — a także relację przynależności pola do tabeli. Podgraf przepływu danych między tabelami a zadaniami jest *dwudzielny* i skierowany, co pozwala interpretować go jako graf skierowany acykliczny (DAG). W niniejszej pracy przedmiotem detekcji są wyłącznie węzły reprezentujące tabele.

Rysunek 2.1 przedstawia poglądowy fragment grafu przepływu danych przed usunięciem zadania i po jego usunięciu. Usunięcie zadania J1 pozbawia tabele T1 i T2 konsumenta, a tabelę T3

— producenta; wszystkie trzy stają się zainfekowane, choć pozostają w grafie (T3 zachowuje połączenie z zadaniem J2).



RYSUNEK 2.1: Fragment grafu przepływu danych: (a) graf oryginalny, (b) graf po usunięciu zadania J1. Tabele zainfekowane oznaczono kolorem czerwonym, krawędzie utracone — linią przerywaną.

Wszystkie eksperymenty oparto na zbiorze danych DLG-DG-23; jego pełną charakterystykę — konkretną reprezentację typów węzłów i krawędzi, anonimizację oraz statystyki 18 grafów — zebrano w sekcji 3.2.

2.3 Niekompletność grafów lineage — broken lineage

Graf lineage bywa niekompletny z kilku powodów. Tabele tymczasowe, tworzone na potrzeby pojedynczej sesji lub procesu ETL, znikają po jego zakończeniu. Jeśli pośredniczyły w przepływie danych między dwoma trwałymi tabelami, znika wraz z nimi odpowiedni fragment grafu. Analogiczny skutek mają widoki zmaterializowane tworzone tymczasowo oraz funkcje definiowane przez użytkownika (ang. *User-Defined Function*, UDF), które opakowują logikę transformacji bez jawnego zapisu przepływu danych [13].

Zjawisko to, określane jako *broken lineage*, sprawia, że tabela traci część lub całość informacji o swoim pochodzeniu: przestaje mieć widocznego producenta danych (zadanie zapisujące) lub ich konsumenta (zadanie czytające), mimo że rzeczywista zależność istnieje. W niniejszej pracy problem broken lineage jest modelowany jako zadanie *detekcji zainfekowanych węzłów* — dla każdej tabeli algorytm ocenia prawdopodobieństwo, że jej lineage został zerwany. Formalne ujęcie problemu przedstawiono w rozdziale 3.

2.4 Detekcja węzłów o zerwanym lineage

Detekcja anomalii na poziomie węzłów grafu polega na przypisaniu każdemu węzłowi wyniku (score) wyrażającego stopień, w jakim odbiega on od wzorca uznawanego za poprawny, a następnie na uszeregowaniu węzłów według tego wyniku. W kontekście grafów lineage węzłem podejrzanym jest tabela, której sąsiedztwo w grafie DAG sugeruje niekompletność pochodzenia danych.

W przeciwieństwie do predykcji krawędzi, zadanie nie polega na wskazaniu *której* krawędzi brakuje, lecz *które* tabele są dotknięte brakiem. Jest to zadanie rankingowe, oceniane metrykami czułymi na uszeregowanie (sekcja 2.5), a nie zadanie klasyfikacji przy ustalonym progu. Motywacja

dla tego ujęcia wynika z ograniczeń danych DLG-DG-23: rzadkość grafów sprawia, że sygnał o konkretnej brakującej parze obiektów jest zbyt słaby (sekcja 4.1), natomiast wskazanie podejrzanych tabel pozostaje wykonalne.

W pracy zastosowano trzy klasy metod detekcji:

1. **Heurystyki strukturalne** — reguły oparte wyłącznie na pozycji węzła w grafie (stopień, rola korzeni/liść) oraz autorski score kompletności *peer-consistency*; nie wymagają uczenia.
2. **Klasyczne metody ML** — Random Forest, RUSBoost i LightGBM; klasyfikują węzeł na podstawie wektora cech pozycji w DAG. Ze względu na anonimizację (UUID) metryki podobieństwa nazw (np. Jaro-Winkler stosowane w [2]) są nieaplikowalne.
3. **Autorski detektor LineageDetector** — łączy cechy pozycji w DAG, relacyjny score kompletności oraz kalibrację prawdopodobieństw (isotonic); jego wynikiem jest oszacowanie $p(\text{zainfekowana})$.

2.5 Metryki ewaluacji

Zadanie detekcji ma charakter rankingowy, dlatego stosuje się metryki oceniające jakość uszeregowania węzłów, a nie pojedynczej decyzji przy progu:

- **AUC-ROC** — pole pod krzywą ROC; mierzy zdolność modelu do postawienia zainfekowanej tabeli wyżej w rankingu niż czystej,
- **AUC-PR** — pole pod krzywą precyzja–czułość; szczególnie istotne przy niezrównoważonych klasach, gdy liczba tabel czystych wielokrotnie przewyższa liczbę zainfekowanych,
- **Precision@k** — frakcja tabel faktycznie zainfekowanych wśród k najwyżej sklasyfikowanych,
- **Recall@k** — frakcja wszystkich zainfekowanych tabel odzyskanych w pierwszych k pozycjach rankingu,
- **Hits@k** — czy wśród k czołowych pozycji znajduje się co najmniej jedna tabela zainfekowana,
- **Brier score** — jakość kalibracji prawdopodobieństw (istotna dla detektora zwracającego $p(\text{zainfekowana})$).

Metryki rankingowe są szczególnie istotne w warunkach niezrównoważonych klas, charakterystycznych dla grafów rzadkich, gdzie liczba tabel czystych wielokrotnie przewyższa liczbę tabel zainfekowanych [2, 8].

2.6 Wyniki systematycznego przeglądu literatury

Przeprowadzony systematyczny przegląd literatury (ang. *Systematic Literature Review*, SLR) pozwolił zidentyfikować prace bezpośrednio odnoszące się do postawionych pytań badawczych. Wszystkie dotychczasowe prace o broken lineage między obiektami bazy modelują problem jako *predykcję krawędzi*; detekcji na poziomie węzłów — wskazania, które tabele mają zerwany lineage — literatura ta nie podejmuje. Tę lukę wypełnia niniejsza praca.

Omówione prace pogrupowano w dwie kategorie: prace *bazowe*, stanowiące bezpośredni fundament niniejszej pracy (metoda kontrastowa oraz wykorzystany zbiór danych referencyjnych), a następnie *metody pokrewne* odkrywania zależności lineage, wyznaczające szerszy kontekst metodyczny.

2.6.1 Prace bazowe: metoda kontrastowa i zbiór danych

Leveraging machine learning techniques for discovering broken lineage links

Autorzy: Boiński, Andrzejewski, Grocholewski, Gruszczyński, Wrembel [2].

Problem badawczy. Ciągłość grafów *data lineage* ulega przerwaniu, gdy w potokach ETL wykorzystywane są obiekty pośrednie — tabele tymczasowe lub UDF. W rezultacie narzędzia klasy *data governance* tracą ślad przepływu danych i błędnie interpretują trwałą tabelę źródłową oraz tabelę docelową jako byty niezależne.

Zaproponowane rozwiązanie. Autorzy formułują problem rekonstrukcji przerwanych zależności jako binarną klasyfikację par obiektów bazodanowych, z inżynierią cech opartą na metadanych i podobieństwie nazw (Jaro–Winkler), klasyfikatorami Random Forest, RUSBoost, BalancedBagging i LightGBM oraz generowaniem etykiet metodą rekurencyjnego zastępowania tabel tymczasowych ich trwałymi źródłami. Ze względu na brak publicznych danych autorzy wygenerowali syntetyczne schematy baz LLM-ami.

Analiza krytyczna i relacja do pracy. Najwyższą skuteczność osiągnął RUSBoost ($F1 = 0,79$). Artykuł ten jest punktem wyjścia niniejszej pracy, która jednak *istotnie się od niego różni*: Boiński et al. przewidują *krawędzie* i korzystają z semantyki nazw, podczas gdy w niniejszej pracy detekcja odbywa się na poziomie *węzłów* i wyłącznie na podstawie topologii (dane DLG-DG-23 są zanonimizowane). Adaptowano natomiast rodzinę klasyfikatorów (RF, RUSBoost, LightGBM) jako punkty odniesienia dla autorskiego detektora.

An open dataset of data lineage graphs

Autorzy: Chen, Zhao, Li, Zhang, Long, Zhou [4].

Problem badawczy. Rozwój metod w obszarze *data governance* jest ograniczany przez brak ogólnodostępnych, rzeczywistych zbiorów danych; przedsiębiorstwa nie udostępniają grafów lineage ze względu na poufność.

Zaproponowane rozwiązanie. Autorzy opublikowali zbiór DLG-DG-23 — pierwszy otwarty zestaw rzeczywistych grafów pochodzenia danych: 18 grafów w zakresie 278–17 085 węzłów, ze środowiska produkcyjnego Huawei Cloud (dane zanonimizowane), z węzłami trzech typów (tabela, zadanie, pole) i relacjami przepływu oraz hierarchii.

Analiza krytyczna i relacja do pracy. Wartość zbioru wynika z jego rzeczywistego, przemysłowego pochodzenia; ograniczeniem jest niewielka liczba grafów oraz anonimizacja atrybutów. W niniejszej pracy DLG-DG-23 jest referencyjnym zbiorem danych (*ground truth*) we wszystkich eksperymentach.

2.6.2 Metody pokrewne odkrywania zależności lineage

Data lineage discovery in databases based on knowledge graph link prediction

Autorzy: Dutkiewicz, Misiołek, Wrembel [6].

Problem badawczy. Automatyczne odkrywanie brakujących zależności lineage między obiektami bazy danych jest słabo zbadane, ponieważ rzeczywiste dane biznesowe i schematy baz nie są publicznie dostępne — w konsekwencji brakuje benchmarków umożliwiających rzetelną ewaluację metod odkrywania.

Zaproponowane rozwiązanie. Praca wnosi trzy artefakty: (1) procedurę generowania scenariuszy broken lineage za pomocą frameworku opartego na LLM, obejmującą dziewięć typów scenariuszy (m.in. kopiowanie danych, transformacje, złączenia, deduplikację, widoki, zapytania rekuren-

cyjne, partycjonowanie i tabele tymczasowe); (2) model semantyczny transformujący scenariusze do grafu wiedzy (RDF, relacja *valueDerivedFrom* inspirowana ontologią PROV-O); (3) dostosowany do lineage protokół ewaluacji w pełni indukcyjnej predykcji krawędzi ($\mathcal{E}_{\text{train}} \cap \mathcal{E}_{\text{test}} = \emptyset$) z użyciem path-based GNN, próbkowaniem negatywów 50:1 oraz metrykami AUROC i Hits@10.

Analiza krytyczna i relacja do pracy. Wyniki są silnie zależne od typu scenariusza: proste kopiowanie danych osiąga AUROC 0,92 (model dedykowany), lecz scenariusze złożone spadają do $\approx 0,60$, a *tabele tymczasowe* — najczęstsza praktyczna przyczyna broken lineage — do AUROC 0,49, czyli poziomu losowego. Potwierdza to trudność problemu nawet dla zaawansowanych metod grafowo-semantycznych. Z pracy tej zaadaptowano do niniejszej pracy *protokół indukcyjny cross-graph* oraz zestaw metryk rankingowych. Zasadnicza różnica: Dutkiewicz et al. przewidują krawędzie w grafie wiedzy wzbogaconym semantycznie, podczas gdy niniejsza praca operuje na poziomie węzłów i czystej topologii.

On discovering provenance of data warehouse objects created by select-project queries

Autorzy: Andrzejewski, Boiński, Wrembel [1].

Problem badawczy. W hurtowniach danych obiekty (np. widoki zmaterializowane) powstają z innych obiektów przez zapytania select-project, a potoki integracji danych tworzą obiekty tymczasowe i ulotne (tabele tymczasowe, zmienne globalne, kursory), których zależności nie są automatycznie rejestrowane i zanikają po zakończeniu zadania.

Zaproponowane rozwiązanie. Podejście analityczne (niewymagające uczenia maszynowego): (1) algorytm odzyskujący współczynniki (nachylenie i wyraz wolny) funkcji liniowej zastosowanej w projekcji oraz (2) algorytm odtwarzający warunek selekcji użyty w zapytaniu. Zależność między obiektami jest wykrywana przez dopasowanie modelu matematycznego do *wartości* przechowywanych danych.

Analiza krytyczna i relacja do pracy. Metoda daje dokładne, interpretowalne wyniki, lecz jej założenia są restrykcyjne: wymaga dostępu do zawartości tabel (wartości danych) oraz ogranicza się do transformacji liniowych w zapytaniach select-project. W warunkach niniejszej pracy (zanonimizowane grafy bez dostępu do danych i schematów) podejście to jest nieaplikowalne. Ograniczenie to uzasadnia pytanie postawione w niniejszej pracy: jak daleko sięga sygnał czysto topologiczny.

Inferring missing data lineage links from schema metadata using transformer-based models

Autorzy: Brzeski, Roman [3].

Problem badawczy. Istniejące metody ekstrakcji lineage (instrumentacja, analiza czasu wykonania, adnotowane potoki) zawodzą, gdy informacja o przepływie jest częściowo niedostępna — np. w środowiskach hybrydowych, systemach zastanych (*legacy*) lub przy audytach post hoc.

Zaproponowane rozwiązanie. Dwustopniowa metoda wnioskowania *kierunkowych* zależności lineage na poziomie kolumn wyłącznie z metadanych schematu, inspirowana technikami dopasowywania schematów: bi-enkoder transformerowy filtruje kandydatów, a cross-enkoder wykonuje precyzyjną predykcję. Ewaluacja na wielkoskalowych danych komercyjnych z ekstremalną nierównowagą klas (1 poprawny link na 90 000 par kandydackich).

Analiza krytyczna i relacja do pracy. Praca pokazuje, że przy dostępnej semantyce nazw kolumn i schematów modele transformerowe skutecznie odzyskują zależności mimo skrajnej nierównowagi klas. Warunkiem jest jednak dokładnie ta informacja, której w zbiorze DLG-DG-23

brak (identyfikatory UUID) — sygnał semantyczny i sygnał topologiczny badany w niniejszej pracy są zatem komplementarne, a ich połączenie jest naturalnym kierunkiem dalszych badań.

2.6.3 Luka badawcza

Wszystkie omówione prace — od klasycznego ML na cechach semantyczno-topologicznych [2], przez metody analityczne [1] i transformery na metadanych schematu [3], po indukcyjne GNN na grafach wiedzy [6] — modelują broken lineage jako odtwarzanie *zależności* (krawędzi lub parametrów transformacji). Żadna nie stawia pytania, *które tabele mają zerwany lineage*. Niniejsza praca uzupełnia ten obraz: wskazanie podejrzanych tabel może poprzedzać kosztowniejsze odtwarzanie konkretnych zależności.

2.6.4 Prace kontekstowe

Spośród pozostałych prac na uwagę zasługuje metoda *Provenance Graph Kernel* (PGK) [8], definiująca miary podobieństwa grafów proveniencji. Detekcja anomalii na poziomie węzłów pojawia się w pokrewnej dziedzinie systemowych grafów provenance i bezpieczeństwa (np. heterogeniczne GNN [12]), nie była jednak dotąd stosowana do lineage obiektów bazodanowych. Prace dotyczące proveniencji rekurencyjnych zapytań SQL [5], analizy logów [11] oraz UDF [13] tworzą szerszy kontekst literaturowy.

Rozdział 3

Problem i sposób jego rozwiązania

Rozdział precyzuje formalną definicję problemu detekcji węzłów o zerwanym lineage (sekcja 3.1). Sekcja 3.2 charakteryzuje zastosowany zbiór danych, sekcja 3.3 — metodykę symulacji broken lineage przez usuwanie jobów, a sekcja 3.4 opisuje architekturę potoku przetwarzania. Szczegóły implementacji metod detekcji omówiono w sekcji 3.5.

3.1 Sformułowanie problemu

Graf lineage modeluje się jako skierowany graf DAG przepływu danych $G = (V, E)$, gdzie $V = V_J \cup V_T \cup V_F$ obejmuje węzły typu zadanie (*Data Job*), tabela (*Data Table*) i pole, a E zawiera krawędzie przepływu danych (*DATA_FLOW*) między jobami a tabelami. Zjawisko broken lineage odpowiada usunięciu z grafu pewnego zadania J : tabele z nim incydentne — jego poprzedniki (tabele karmiące J) oraz następniki (tabele karmione przez J) — tracą część informacji o swoim pochodzeniu i stają się *zainfekowane* (ang. *infected*).

Obserwowany graf ma postać $G_{\text{obs}} = G \setminus \{J_1, \dots, J_m\}$, powstałą przez usunięcie podzbioru zadań. Zadaniem detekcji jest wyznaczenie funkcji, która każdej tabeli $t \in V_T$ przypisuje wynik $s(t) \in [0, 1]$ interpretowany jako oszacowane prawdopodobieństwo p (tabela t jest zainfekowana). Wynikiem nie jest decyzja binarna przy ustalonym progu, lecz *ranking* tabel — tabele o najwyższym wyniku są pierwszymi kandydatami do ręcznej inspekcji.

Etykiety referencyjne (*ground truth*) generowane są automatycznie: tabela jest zainfekowana wtedy i tylko wtedy, gdy była incydentna do co najmniej jednego usuniętego zadania. Zbiór kandydatów ograniczono do węzłów typu *Data Table* — węzły typu pole (*field*) łączy z tabelami wyłącznie relacja *PARENT_CHILD* i nie podlegają one detekcji.

Zasadniczą trudność zadania polega na tym, że osierocona tabela-następnik po usunięciu zadania wygląda jak prawdziwa tabela źródłowa ($\text{in_df} = 0$). Rozróżnienie tych dwóch przypadków wymaga cech opisujących pozycję węzła w grafie DAG oraz uczenia *indukcyjnego* (cross-graph), a nie samego stopnia węzła.

3.2 Zbiór danych DLG-DG-23

Zastosowany zbiór danych pochodzi z pracy Chen et al. [4] i zawiera 18 grafów lineage pobranych ze środowiska produkcyjnego Huawei Cloud. Dane zostały zanonimizowane — nazwy tabel i atrybutów zastąpiono identyfikatorami w formacie UUID, a schematy bazy danych nie zostały udostępnione. Zbiór opublikowano w 2024 roku jako pierwszy otwarty zestaw rzeczywistych grafów lineage.

Każdy graf zawiera węzły trzech typów — **Data Job** (zapytanie SQL lub zadanie ETL, ang. *Extract, Transform, Load*), **Data Table** (tabela lub widok przechowujący dane) oraz **field** (atrybut/kolumna należąca do tabeli) — połączone krawędziami dwóch typów: **DATA_FLOW** (skierowana krawędź przepływu danych między tabelą a zadaniem) oraz **PARENT_CHILD** (relacja przynależności pola do tabeli). Pełny graf jest zatem *heterogeniczny*. Przedmiotem detekcji są wyłącznie węzły typu *Data Table*, a sygnał czerpany jest z podgrafu przepływu danych (krawędzie *DATA_FLOW* między węzłami *Data Table* i *Data Job*); podgraf ten ma strukturę dwudzielną i skierowaną, interpretowaną jako graf DAG. Grafy różnią się skalą: liczba węzłów waha się od 278 do 17 085.

Zbiór DLG-DG-23 wybrano, ponieważ jest jedynym publicznie dostępnym zestawem rzeczywistych grafów lineage. Anonimizacja nazw węzłów ma bezpośredni wpływ na dobór metod: metryki podobieństwa oparte na nazwach obiektów (np. Jaro–Winkler stosowana przez Boińskiego et al. [2]) są nieaplikowalne. W niniejszej pracy sygnał ograniczono zatem do atrybutów *topologicznych* grafu. Ograniczenie to jest zarazem częścią pytania badawczego: czy sama struktura grafu lineage wystarcza do wskazania tabel o zerwanym pochodzeniu danych. Metoda skuteczna przy tak ograniczonym wglądzie w graf ma przy tym większą szansę na zastosowanie praktyczne niż podejścia wymagające pełnych metadanych.

3.2.1 Charakterystyka grafów

Tabela 3.1 przedstawia statystyki poszczególnych grafów zbioru DLG-DG-23.

TABELA 3.1: Charakterystyka grafów zbioru DLG-DG-23.

ID	Węzły	Krawędzie DATA_FLOW	Skala
DLG1	298	24	mały
DLG2	464	37	mały
DLG3	603	57	mały
DLG4	415	32	mały
DLG5	1 299	175	średni
DLG6	13 840	1 443	duży
DLG7	574	43	mały
DLG8	546	34	mały
DLG9	756	62	mały
DLG10	5 378	497	duży
DLG11	2 024	121	średni
DLG12	5 645	469	duży
DLG13	2 157	229	średni
DLG14	453	24	mały
DLG15	558	41	mały
DLG16	5 574	745	duży
DLG17	651	38	mały
DLG18	17 085	1 694	duży

Skala: mały < 1 000 węzłów; średni 1 000–5 000; duży > 5 000.

3.3 Metodyka symulacji broken lineage przez usuwanie jobów

Ponieważ zbiór DLG-DG-23 zawiera w pełni zaobserwowane grafy lineage, zjawisko broken lineage symulowane jest przez kontrolowane usuwanie *zadań* (nie pojedynczych krawędzi). Podejście to wiernie odwzorowuje przyczynę broken lineage: zniknięcie tabeli tymczasowej lub ukrycie logiki przez UDF odpowiada utracie całego węzła-zadania wraz z jego krawędziami.

Kandydatami do usunięcia są *joby łączące* — zadania mające co najmniej jedną wejściową i jedną wyjściową krawędź *DATA_FLOW* ($\text{in_df} > 0 \wedge \text{out_df} > 0$). Usunięcie takiego zadania zrywa lineage po obu stronach: tabele karmiące je tracą konsumenta, a tabele przez nie karmione — producenta. Z grafu G budowana jest instancja klasyfikacji węzłów:

1. usuwa się losowy podzbiór jobów łączących (udział regulowany parametrem `removal_ratio`; w eksperymentach przyjęto 10%, a wpływ tego parametru zbadano w analizie wrażliwości w sekcji 4.3),
2. zbiór zainfekowanych tabel to suma tabel incydentnych do usuniętych zadań,
3. graf obserwowany G_{obs} powstaje przez usunięcie wybranych zadań i ich krawędzi,
4. kandydatami są wszystkie tabele w G_{obs} ; etykieta 1 oznacza tabelę zainfekowaną, etykieta 0 — czystą.

Dodatkowo zaimplementowano wariant kontrolny, w którym każda instancja usuwa dokładnie *jedno* zadanie, co daje czystą interpretację przyczynową (jedno zdarzenie broken lineage = jeden brakujący job).

3.3.1 Kontrola przecieku: wykluczenie węzłów izolowanych

Usunięcie zadania może pozbawić tabelę *wszystkich* krawędzi przepływu danych ($\text{in_df} = 0 \wedge \text{out_df} = 0$). Ponieważ w oryginalnych grafach tabele izolowane nie występują, trywialna reguła „tabela izolowana $\Rightarrow$ zainfekowana” pozwala rozpoznać znaczną część pozytywów bez żadnego uczenia, sztucznie zawyżając wynik. Aby zapewnić uczciwą ewaluację, wprowadzono flagę `exclude_isolated`, która usuwa z puli kandydatów tabele w pełni izolowane. Wariant ten stawia trudniejsze i bardziej realistyczne zadanie: wykryć tabele o *częściowo* zerwanym lineage, które nadal mają część połączeń. W pracy raportowane są oba warianty, a wariant uczciwy (z wykluczeniem izolowanych) traktowany jest jako główny.

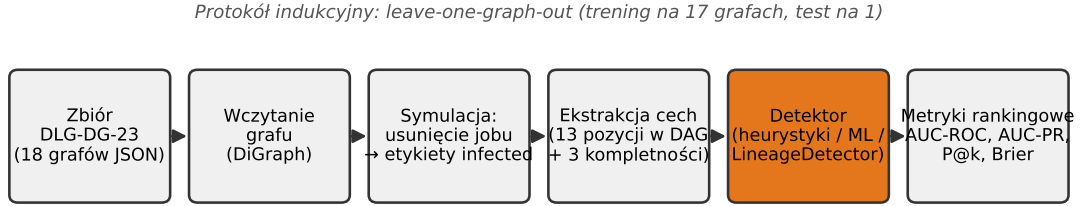
3.4 Architektura rozwiązania

Potok przetwarzania składa się z czterech komponentów: modułu wczytywania grafów, modułu symulacji broken lineage (budowa instancji klasyfikacji węzłów), metod detekcji oraz modułu ewaluacji indukcyjnej.

Rysunek 3.1 przedstawia schemat potoku przetwarzania: od wczytania grafów z plików JSON, przez symulację broken lineage i ekstrakcję cech, po scoring detektora i wyznaczenie metryk rankingowych.

3.4.1 Moduł wczytywania danych

Dane wejściowe każdego grafu przechowywane są w dwóch plikach JSON: `DLGi-node.json` (opisy węzłów z atrybutem `asset_type`) oraz `DLGi-edge.json` (krawędzie z atrybutami `relation_type` i `relation_id`). Moduł `loader.py` buduje z nich obiekt `nx.DiGraph` (NetworkX) i zwraca słownik wszystkich 18 grafów.



RYSUNEK 3.1: Schemat potoku przetwarzania. Każda instancja ewaluacji przebiega w protokole indukcyjnym leave-one-graph-out: model trenowany jest na 17 grafach, a testowany na grafie niewidzianym.

3.4.2 Moduł symulacji (job_removal)

Moduł `job_removal.py` realizuje metodykę z sekcji 3.3: identyfikuje joby łączące, usuwa ich podzbiór, wyznacza zbiór zainfekowanych tabel i zwraca instancję (graf G_{obs} , listę kandydatów, etykiety, listę usuniętych jobów). Obsługuje warianty `side` (strona incydencji liczona jako infected: producenta, konsumenta lub obie) oraz `exclude_isolated`. Funkcja `inductive_split` dzieli identyfikatory grafów na rozłączne zbiory treningowy i testowy (cross-graph).

3.4.3 Moduł ewaluacji (node_metrics)

Moduł `node_metrics.py` wyznacza metryki rankingowe: AUC-ROC, AUC-PR, Precision@k, Recall@k, Hits@k oraz Brier score. Wyniki agregowane są po grafach testowych w protokole indukcyjnym; stosuje się powtórzenia z różnymi ziarnami i trzymowanie wartości skrajnych (min/max) dla stabilności oszacowań.

Ewaluację prowadzi się w protokole *indukcyjnym* cross-graph: model trenuje się na podzbiorze grafów, a testuje na grafach niewidzianych (leave-one-graph-out lub podział zbioru 18 grafów). Zbiory treningowy i testowy są rozłączne, co sprawdza zdolność metody do generalizacji na nowe grafy, a nie jedynie do zapamiętania jednego. Konsekwencją tego protokołu jest wykluczenie metody Node2Vec: uczy ona zanurzeń węzłów transduktywnie, dla konkretnego grafu, przez co embeddingi są geometrycznie nieporównywalne między grafami. Nie jest to luka, lecz *wynik*: motywuje zastosowanie indukcyjnych grafowych sieci neuronowych (GraphSAGE, R-GCN, HGT), wskazanych w rozdziale 5 jako kierunek dalszych badań.

3.5 Zaimplementowane metody detekcji

Zaimplementowano trzy grupy metod detekcji zainfekowanych tabel: heurystyki strukturalne, klasyczne klasyfikatory ML oraz autorski detektor LineageDetector.

3.5.1 Heurystyki strukturalne

Heurystyki strukturalne przypisują tabeli wynik podejrzliwości wyłącznie na podstawie jej pozycji w grafie, bez etapu uczenia. Są naturalnym punktem odniesienia (*baseline*) dla metod uczonych. Miary oparte na wspólnych sąsiadach (Common Neighbors, Jaccard, Adamic-Adar) są w grafie dwudzielnym nieskuteczne, ponieważ węzły tego samego typu nigdy nie mają wspólnych sąsiadów bezpośrednich; wykluczono je z implementacji.

W pracy zaimplementowano cztery heurystyki bazowe:

- **Anomalia stopnia** (`degree_anomaly`) — wynik $1/(1 + \deg(t))$, gdzie $\deg(t)$ jest stopniem całkowitym tabeli. Intuicja: tabela, która utraciła zadanie, ma mniej połączeń niż przed zerwaniem lineage.
- **Brzeg grafu** (`boundary`) — suma wskaźników $\mathbf{1}[\text{in_df} = 0] + \mathbf{1}[\text{out_df} = 0]$. Tabele na brzegu przepływu (korzenie i liście) są podejrzane, ponieważ zerwanie lineage często wypycha tabelę na brzeg grafu.
- **Niska łączność z zadaniami** (`low_job_connectivity`) — wynik $1/(1 + n_J(t))$, gdzie $n_J(t)$ to liczba unikatowych zadań sąsiadujących z tabelą przez krawędzie `DATA_FLOW`.
- **Reguła korzenia** (`rule_root`) — binarny wskaźnik $\mathbf{1}[\text{in_df} = 0]$: każda tabela bez producenta jest uznawana za podejrzaną. To celowo trywialny punkt odniesienia, obrazujący siłę najprostszego możliwego symptomu.

Heurystyki te mierzą wyłącznie *lokalny* symptom (stopień, rola); ich ograniczenie — w szczególności nieodróżnianie osieroconej tabeli od prawdziwego źródła — motywuje metody uczone oraz autorski detektor LineageDetector (sekcja 3.5.3), który wzbogaca opis węzła o relacyjny *score kompletności* (peer-consistency).

3.5.2 Klasyczne metody uczenia maszynowego

Klasyczne metody ML traktują detekcję jako binarną klasyfikację węzła: wejściem jest wektor cech opisujących pozycję tabeli w grafie przepływu danych — to ona, a nie sam stopień, pozwala odróżnić prawdziwą tabelę źródłową od tabeli osieroconej po usunięciu zadania. Ze względu na anonimizację (UUID) jako cechy przyjęto wyłącznie atrybuty topologiczne — metody oparte na nazwach są nieaplikowalne.

Wektor cech węzła

Tabela 3.2 przedstawia pełny wektor 13 cech topologicznych wyznaczanych dla każdej tabeli-kandydata. Cechy opisują pozycję węzła w grafie przepływu danych — to ona, a nie sam stopień, pozwala odróżnić prawdziwą tabelę źródłową od tabeli osieroconej po usunięciu zadania.

Detektor LineageDetector (sekcja 3.5.3) rozszerza ten wektor o trzy cechy kompletności (`peer_producer_score`, `peer_consumer_score`, `completeness_anomaly`), opisane w sekcji 3.5.

Random Forest

Random Forest (RF) to metoda uczenia zespołowego oparta na uśrednianiu predykcji drzew decyzyjnych trenowanych na losowych próbkach danych i cech (*bagging*). Dla obsługi nie zrównoważonych klas zastosowano `class_weight="balanced"`.

RUSBoost

RUSBoost (*Random Under-Sampling Boosting*) łączy boosting AdaBoost z losowym podpróbkowaniem klasy większościowej przed każdą iteracją. Zaimplementowano go dla bezpośredniego nawiązania do wyników Boińskiego et al. [2], przy czym w niniejszej pracy klasyfikuje on węzły, a nie pary obiektów.

TABELA 3.2: Wektor cech topologicznych węzła (moduł `node_features`).

Nr	Cecha	Opis	Rola w detekcji
1	<code>in_df</code>	liczba wejściowych krawędzi <code>DATA_FLOW</code> (od zadań)	brak producenta ($= 0$) to główny symptom zerwania
2	<code>out_df</code>	liczba wyjściowych krawędzi <code>DATA_FLOW</code> (do zadań)	symetryczny symptom: utrata konsumenta
3	<code>degree</code>	stopień całkowity węzła (wszystkie typy krawędzi)	ogólna łączność; spada po usunięciu zadania
4	<code>role</code>	rola w przepływie: źródło / pośrednia / ujście / poza przepływem	pozycja funkcjonalna tabeli w DAG
5	<code>flow_depth</code>	głębokość w DAG (odległość od źródła)	osierocona tabela ma sztucznie małą głębokość
6	<code>flow_reach</code>	zasięg w dół DAG (odległość do ujścia)	symetrycznie: sztucznie mały zasięg
7	<code>depth_plus_reach</code>	suma głębokości i zasięgu	długość ścieżki przez węzeł; wykrywa „skrócone” łańcuchy
8	<code>is_root</code>	wskaźnik $1[\text{in_df} = 0]$	binarna wersja symptomu utraty producenta
9	<code>is_leaf</code>	wskaźnik $1[\text{out_df} = 0]$	binarna wersja symptomu utraty konsumenta
10	<code>n_field_children</code>	liczba pól tabeli (krawędzie <code>PARENT_CHILD</code>)	rozmiar tabeli; sygnał niezależny od przepływu
11	<code>n_job_neighbors</code>	liczba unikatowych zadań sąsiadujących	łączność z warstwą zadań
12	<code>avg_job_in_df</code>	średni stopień wejściowy sąsiednich zadań	charakterystyka otoczenia węzła
13	<code>avg_job_out_df</code>	średni stopień wyjściowy sąsiednich zadań	j.w. — od strony wyjść

LightGBM

LightGBM to implementacja gradientowego boostingu drzew decyzyjnych, budująca kolejne drzewa sekwencyjnie, każde minimalizując błąd reszkowy poprzednika. Parametr `scale_pos_weight` (lub `class_weight`) kompensuje niebalansowanie klas.

3.5.3 Autorski detektor — LineageDetector

Autorski detektor `LineageDetector` (klasa `LineageHealthDetector`) łączy trzy elementy w jeden skalibrowany model:

1. **cechy pozycji w DAG** (z modułu `node_features`) — lokalny opis węzła,
2. **score kompletności peer-consistency** (opisany poniżej) — sygnał relacyjny stanowiący sedno wkładu własnego pracy,
3. **kalibrację prawdopodobieństw** metodą izotoniczną (`CalibratedClassifierCV`) — wyjście jest rzetelnym $p(\text{zainfekowana})$, a nie dowolnym wynikiem.

Score kompletności — peer-consistency. Zaproponowany w pracy score kompletności opiera się na założeniu, że tabela o zerwanym lineage jest *strukturalnie niespójna* ze swoimi sąsiadami (peerami) w grafie DAG. Dla tabeli-korzenia ($\text{in_df} = 0$) rozważa się zbiór peerów-wejść — inne tabele wpadające do tych samych zadań downstream. Jeśli peerzy *mają* producenta, a rozpatrywana tabela nie, to jej brak producenta jest podejrzany: prawdziwe źródło danych sąsiaduje zwykle z innymi źródłami. Symetrycznie dla tabeli-liścia ($\text{out_df} = 0$) i jej peerów po stronie konsumpcji.

Formalnie dla tabeli t definiuje się:

$$\text{prod}(t) = \frac{1}{|P(t)|} \sum_{p \in P(t)} \mathbf{1}[\text{in_df}(p) > 0], \quad \text{cons}(t) = \frac{1}{|C(t)|} \sum_{p \in C(t)} \mathbf{1}[\text{out_df}(p) > 0],$$

gdzie $P(t)$ i $C(t)$ oznaczają zbiory peerów odpowiednio po stronie producenta i konsumenta, a $\mathbf{1}[\cdot]$ — funkcję wskaźnikową. Bramkowany sygnał anomalii aktywuje się wyłącznie na brzegu grafu:

$$\text{anomaly}(t) = \max(\mathbf{1}[\text{in_df}(t) = 0] \cdot \text{prod}(t), \mathbf{1}[\text{out_df}(t) = 0] \cdot \text{cons}(t)).$$

Sygnał ten ma charakter relacyjny — jest jakościowo różny od lokalnego stopnia węzła i dostarcza informacji niedostępnej klasyfikatorowi korzystającemu wyłącznie z cech lokalnych. Trzy wynikające z niego cechy (`peer_producer_score`, `peer_consumer_score`, `completeness_anomaly`) rozszerzają wektor cech z tabeli 3.2 i są dostępne wyłącznie detektorowi LineageDetector.

Bazowym estymatorem jest Random Forest (domyślnie; wariant alternatywny — LightGBM). Interfejs `fit/score` jest zgodny z pozostałymi klasyfikatorami, co umożliwia bezpośrednie porównanie w tym samym potoku. Konstrukcja detektora umożliwia badania ablacyjne: wyłączenie kalibracji (`calibrate=False`) izoluje jej wpływ na Brier score, a wyłączenie cech kompletności (`use_completeness=False`) — wpływ sygnału relacyjnego na AUC.

3.6 Dostępność implementacji i pochodzenie algorytmów

Całość implementacji opisanej w niniejszej pracy — moduły wczytywania danych, symulacji broken lineage, ekstrakcji cech, metod detekcji oraz ewaluacji, wraz z testami jednostkowymi i skryptami odtwarzającymi eksperymenty z rozdziału 4 — udostępniono w publicznym repozytorium Git pod adresem <https://github.com/uzupelnic>¹. Repozytorium zawiera instrukcję uruchomienia oraz pliki konfiguracyjne biegu kanonicznego, co zapewnia odtwarzalność raportowanych wyników.

Metody z grupy heurystyk oraz klasyczne klasyfikatory nie są oryginalnym wkładem pracy — stanowią *punkty odniesienia* (baseline) zaczerpnięte z literatury i z publicznych bibliotek, a wkładem własnym jest ich *adaptacja* do zadania detekcji zainfekowanych węzłów w grafie lineage oraz autorski detektor LineageDetector. Pochodzenie i zakres adaptacji poszczególnych składników zestawiono poniżej:

- **Heurystyki strukturalne** — inspirowane rodziną miar predykcji krawędzi w grafach (Common Neighbors, Jaccard, Adamic-Adar, Preferential Attachment) [8]. Ponieważ graf przepływu danych jest dwudzielny, miary oparte na wspólnych sąsiadach są w nim nieskuteczne; zaadaptowano je do postaci *scoringu pojedynczego węzła* opartego na stopniu i roli w DAG.
- **Random Forest** — implementacja z biblioteki `scikit-learn` [10]. Adaptacja: wejściem jest autorski wektor cech topologicznych węzła (tab. 3.2) zamiast cech pary obiektów; niezbalansowanie klas obsłużono przez `class_weight`.
- **RUSBoost** — implementacja z biblioteki `imbalanced-learn` [9], wybrana dla bezpośredniego nawiązania do pracy Boińskiego et al. [2]. Adaptacja: klasyfikacja *węzła* zamiast pary, na cechach topologicznych.
- **LightGBM** — biblioteka gradientowego boostingu firmy Microsoft [7]. Adaptacja: strojenie `scale_pos_weight` pod rozkład klas w zadaniu detekcji węzłów.

¹Dostęp zweryfikowano w dniu obrony pracy.

- **LineageDetector** — *wkład własny*: łączy powyższe cechy z autorskim relacyjnym score kompletności peer-consistency (sekcja 3.5.3) i kalibracją izotoniczną; nie ma bezpośredniego odpowiednika w literaturze.

Takie rozgraniczenie — komponenty zapożyczone z jawnie wskazanym źródłem kontra elementy autorskie — pozwala jednoznacznie ocenić rzeczywisty wkład pracy i odróżnić go od narzędzi ogólnego przeznaczenia.

Rozdział 4

Ocena rozwiązania

W rozdziale przedstawiono wyniki eksperymentów przeprowadzonych zgodnie z metodyką opisaną w rozdziale 3. Dla każdej z metod detekcji podano wartości metryk rankingowych uśrednione po grafach zbioru DLG-DG-23 w protokole indukcyjnym.

- 4.1 Badanie wstępne: predykcja krawędzi
- 4.2 Wyniki detekcji węzłów o zerwanym lineage
- 4.3 Kontrola przecieku i analiza wrażliwości
- 4.4 Analiza wyników

Rozdział 5

Uwagi końcowe

Rozdział podsumowuje uzyskane wyniki i wskazuje kierunki dalszych badań.

5.1 Podsumowanie

5.2 Kierunki dalszych badań

Literatura

- [1] Witold Andrzejewski, Paweł Boiński, and Robert Wrembel. On Discovering Provenance of Data Warehouse Objects Created by Select-Project Queries. In *Proceedings of the Workshops of the EDBT/ICDT 2026 Joint Conference*, CEUR Workshop Proceedings, Tampere, Finland, 2026. CEUR-WS.org.
- [2] Paweł Boiński, Witold Andrzejewski, Miłosz Grochowski, Tobiasz Gruszczyński, and Robert Wrembel. Leveraging machine learning techniques for discovering broken lineage links between database objects. In *Proceedings of the 33rd International Conference on Information Systems Development (ISD2025)*, Belgrade, Serbia, 2025.
- [3] Maciej Brzeski and Adam Roman. Inferring Missing Data Lineage Links from Schema Metadata Using Transformer-Based Models. In *Proceedings of the VLDB 2025 Workshop: Applied AI for Database Systems and Applications (AIDB 2025)*. VLDB Endowment, 2025.
- [4] Yunpeng Chen, Ying Zhao, Xuanjing Li, Jiang Zhang, Jiang Long, and Fangfang Zhou. An open dataset of data lineage graphs for data governance research. *Visual Informatics*, 8:1–5, 2024.
- [5] Benjamin Dietrich, Tobias Müller, and Torsten Grust. Data Provenance for Recursive SQL Queries. In *Proceedings of the 14th Workshop on the Theory and Practice of Provenance (TaPP '22)*, Philadelphia, PA, USA, 2022. ACM.
- [6] Jakub Dutkiewicz, Paweł Misiorek, and Robert Wrembel. Data Lineage Discovery in Databases Based on Knowledge Graph Link Prediction. In *Proceedings of the Workshops of the EDBT/ICDT 2026 Joint Conference*, CEUR Workshop Proceedings, Tampere, Finland, 2026. CEUR-WS.org.
- [7] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.
- [8] David Kohan Marzagão, Trung Dong Huynh, Ayah Helal, Sean Baccas, and Luc Moreau. Provenance Graph Kernel. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2023.
- [9] Guillaume Lemaître, Fernando Nogueira, and Christos K. Aridas. Imbalanced-learn: A Python Toolbox to Tackle the Curse of Imbalanced Datasets in Machine Learning. *Journal of Machine Learning Research*, 18(17):1–5, 2017.
- [10] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [11] Fotis Psallidas, Ashvin Agrawal, Chandru Sugunan, Khaled Ibrahim, Konstantinos Karanasos, Jesús Camacho-Rodríguez, Avriella Floratou, Carlo Curino, and Raghuram Ramakrishnan. ONEPROVENANCE: Efficient Extraction of Dynamic Coarse-Grained Provenance From Database Query Event Logs. *CoRR*, abs/2210.14047, 2022. Technical Report; deployed in Microsoft Purview.

- [12] Dibin Shan, Xuehui Du, Wenjuan Wang, Na Wang, and Aodi Liu. KPI-HGNN: Key Provenance Identification Based on a Heterogeneous Graph Neural Network for Big Data Access Control. *Information Sciences*, 2023.
- [13] Masaya Yamada, Hiroyuki Kitagawa, Toshiyuki Amagasa, and Akiyoshi Matono. Augmented lineage: traceability of data analysis including complex UDF processing. *The VLDB Journal*, 32:963–983, 2023.



© 2026 Maria Nowicka

Instytut Informatyki, Wydział Informatyki i Telekomunikacji
Politechnika Poznańska

Skład przy użyciu systemu $\text{\LaTeX}$ na platformie Overleaf.